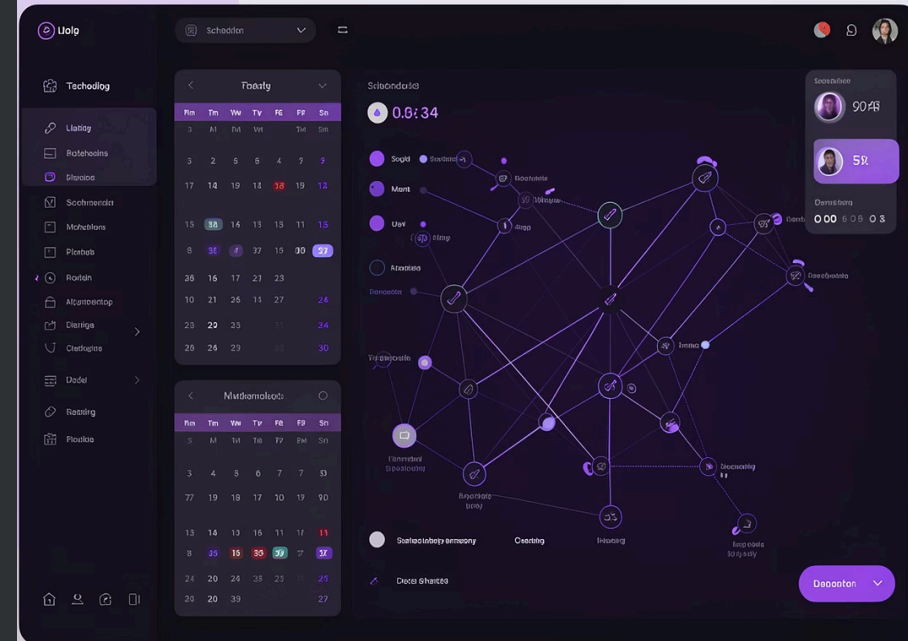


SmartScheduler

Système intelligent de génération d'emplois du temps

Gao Ewann · Hessani Nour · Benaga Mohammed-Ali · Bruno Matteo

DIA2



Problématique

Générer un emploi du temps universitaire automatiquement.

Le défi :

- Affecter chaque cours à un créneau, une salle, un enseignant
- Respecter simultanément toutes les contraintes (prof disponible, salle adaptée, capacité, etc.)
- Problème NP-complet — l'espace de recherche explose

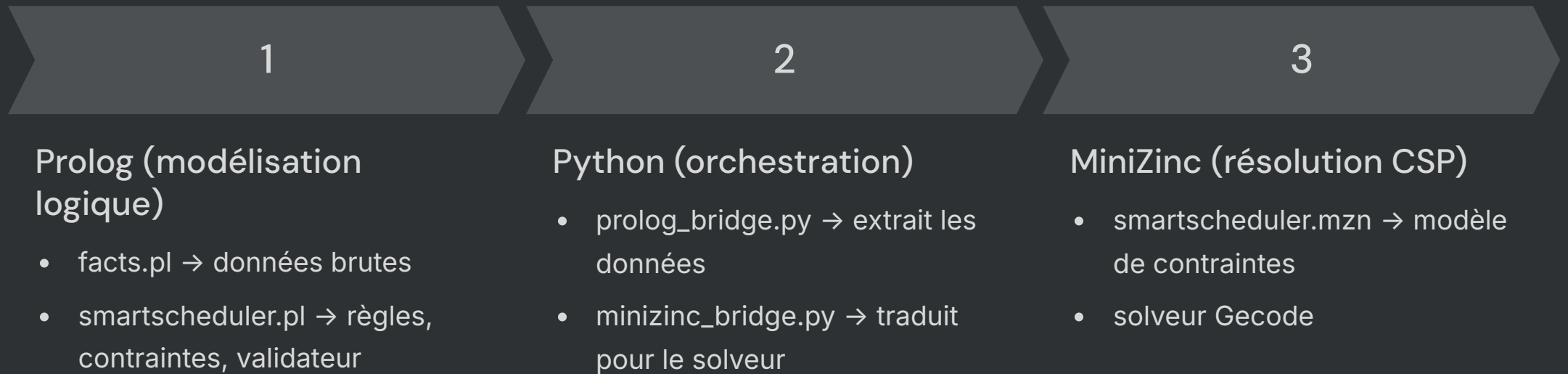
Petit calcul : $3 \text{ cours} \times 2 \text{ profs} \times 2 \text{ salles} \times 4 \text{ créneaux} = 48 \text{ combinaisons}$.

Avec 20 cours, 10 profs, 15 salles, 30 créneaux → des milliards de combinaisons.

Notre approche : programmation logique + solveur par contraintes.

Architecture du système

Pipeline en 3 couches :



Et au-dessus : app.py — interface web Streamlit pour visualiser.

Modélisation Prolog

facts.pl — la base de faits :

```
cours(info201).    prof(martin).  
cours(info202).    prof(dupont).  
salle(amphi1).     groupe(g1).  
salle(lab3).        groupe(g2).  
  
prerequis(info202, info201).  
capacite(amphi1, 120).  
peut_enseigner(martin, info201).  
dispo(martin, c1).  
creneau(c1, lundi, 10, 12).
```

smartscheduler.pl — les règles dérivées :

- Prerequis transitifs ($A \rightarrow B \rightarrow C$)
- Détection de cycles
- 8 prédicats de validation (conflits prof/salle/groupe, capacité, type, dispo...)

prolog_bridge.py

Objectif : transformer la base Prolog en dictionnaires Python.



1. Charger

```
prolog.consult("smartscheduler  
.pl")
```



2. Interroger avec des requêtes

```
requete("cours(X)")      #  
liste des cours  
requete("prerequis(X, Y)")  #  
couples (cours, prereq)  
requete("dispo(P, C)")     #  
couples (prof, créneau)
```



3. Agréger toutes les requêtes

dans un dictionnaire `donnees` que MiniZinc pourra consommer.

Le modèle MiniZinc

smartscheduler.mzn — le modèle de contraintes.

Variables de décision (pour chaque séance) :

- `profSeance[s]` — quel prof
- `salleSeance[s]` — quelle salle
- `creneauSeance[s]` — à quel moment

(Le groupe et le cours sont fixés en entrée.)

Contraintes dures :

- Prof habilité à enseigner ce cours
- Prof disponible sur le créneau
- Salle compatible avec le type de cours
- Capacité salle $\geq$ effectif du groupe
- Pas de chevauchement prof / salle / groupe au même créneau

Mode actuel : `solve satisfy` — recherche d'une solution valide.

minizinc_bridge.py

Objectif : envoyer les données Python au solveur.

4 étapes :

01

Indexer

convertir noms en entiers (info201 → 1, martin → 1...) car MiniZinc travaille avec des nombres.

03

Lancer Gecode via l'API Python

```
instance = Instance(solver, model)
result = instance.solve()
```

02

Construire les matrices

```
profPeutFaire = [| 1, 1, 0 ← martin
                  | 0, 0, 1 |] ← dupont
```

04

Reconvertir les indices en noms

pour produire le planning final lisible.

Interface web Streamlit

app.py — visualisation interactive du planning.

Lancement local :

```
streamlit run app.py
```

3 onglets :

1

Données

- stats de la base
- prérequis
- créneaux
- habilitations

2

Planning

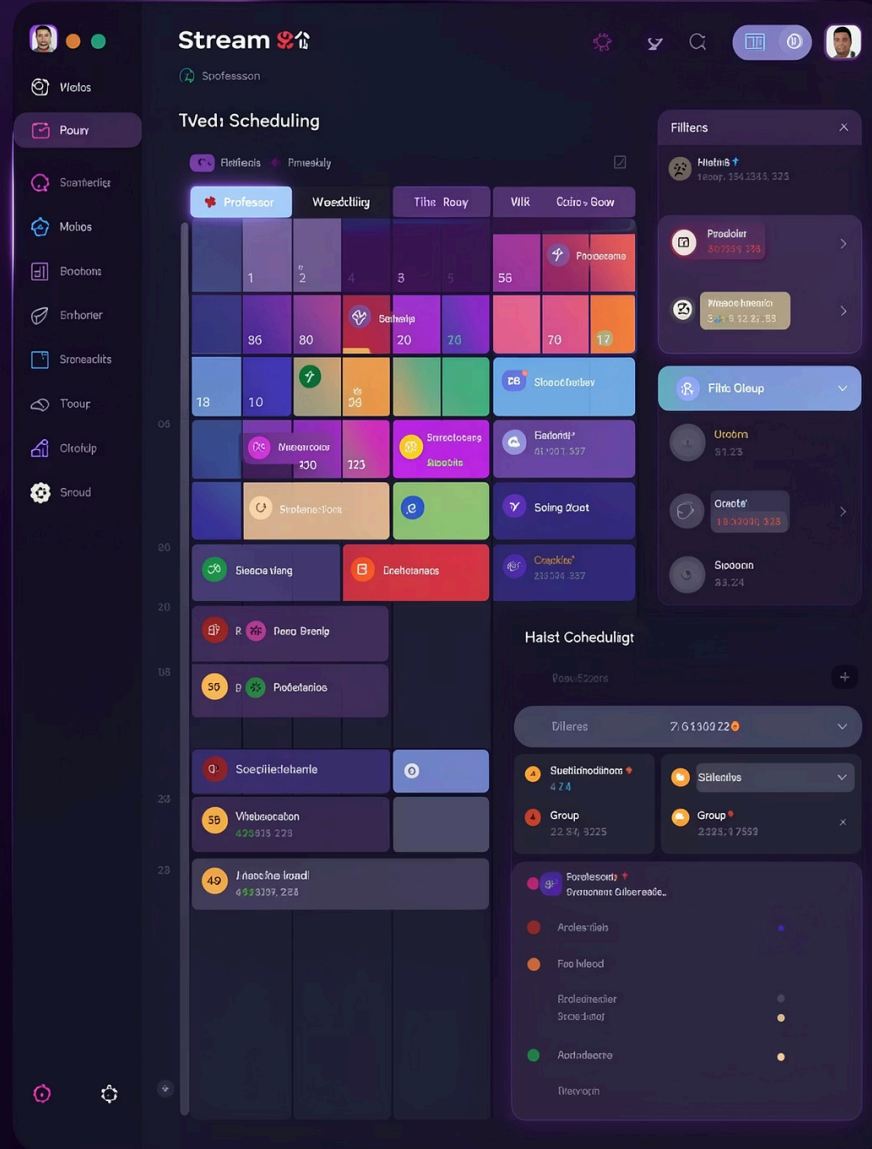
- génération en un clic
- grille hebdomadaire colorée par groupe
- vues filtrées (prof / salle / groupe)
- indicateurs de qualité

3

Export

- téléchargement CSV

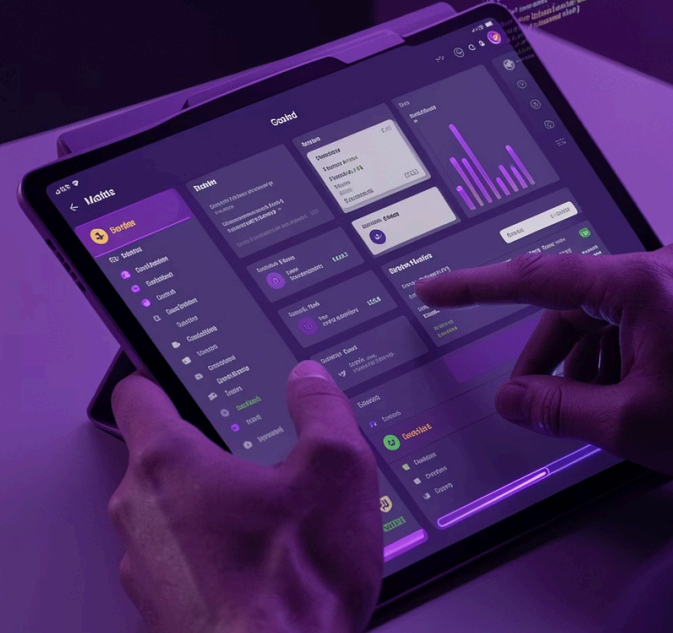
Stack : Streamlit + pandas, thème sombre custom (palette indigo).



Démonstration

Démonstration live

(Passage à l'app Streamlit dans le navigateur)



Tests & validation

Tests unitaires sur la base Prolog (tests/test_scheduler.pl) :

- Prérequis directs et transitifs
- Détection de cycles
- Absence de conflits prof / salle / groupe
- Capacité, type de salle, disponibilités, habilitations

Validation interactive en console :

```
?- planning_valide.  
true.
```

```
?- afficher_planning.  
=== Groupe g1 ===  
info201 | lundi 10h-12h | martin | amphi1  
info202 | mardi 10h-12h | martin | a101  
info203 | lundi 14h-16h | dupont | lab3
```

Limites & perspectives

Limites actuelles :

- Pas d'optimisation multicritère (mode satisfy uniquement)
- Pas de gestion multi-semestre
- Instances de test de petite taille
- Dépend de SWI-Prolog et MiniZinc installés localement

Pistes d'amélioration :

- Fonction objectif pondérée (minimiser les trous, équilibrer la charge, mercredi après-midi libre)
- Comparaison Gecode / Chuffed / OR-Tools
- Export iCal et PDF
- Déploiement sur Streamlit Cloud

Conclusion

Ce qu'on a réalisé :

- Base de connaissances Prolog cohérente
- Pont Python qui connecte Prolog et MiniZinc
- Génération automatique d'emplois du temps valides
- Interface web interactive en mode sombre

Ce qu'on a appris :

- Programmation logique (Prolog, pyswip)
- Programmation par contraintes (CSP, MiniZinc)
- Intégration multi-langage via Python
- Collaboration sur Git/GitHub

Des questions ?

